Aditya College of Engineering and Technology
Approved by AICTE - New Delhi, Affiliated to VTU - Belagavi
2/2, 2/8, 2/9, Kamakshipura, Hesaraghatta Hobli, Yelahanka, Bengaluru, Karnataka - 560 089

Department of Computer Science & Engineering

ANALYSIS AND DESIGN OF ALGORITHMS LAB

BCSL404

B.E - IV Semester

Lab Manual 2025-26

Name : _____

USN : _____

Batch : _____Section : _____

Aditya College of Engineering and Technology

Approved by AICTE - New Delhi, Affiliated to VTU - Belagavi

2/2, 2/8, 2/9, Kamakshipura, Hesaraghatta Hobli, Yelahanka, Bengaluru, Karnataka - 560 089

Department of Computer Science & Engineering

ANALYSIS AND DESIGN OF ALGORITHMS LAB

Version 1.0

2025 - 2026

Prepared by:

Ms. Barnali Chakraborty.

Assistant Professors,
Dept. of CSE ACET,
Bengaluru

Aditya College of Engineering and Technology
Approved by AICTE - New Delhi, Affiliated to VTU - Belagavi
2/2, 2/8, 2/9, Kamakshipura, Hesaraghatta Hobli, Yelahanka, Bengaluru, Karnataka - 560 089

INSTITUTE VISION

- To create centers of excellence in education and to serve the society by enhancing the quality of life through value based professional leadership.

INSTITUTE MISSION

- To provide high quality technical and professionally relevant education in a diverse learning environment.
- To provide the values that prepare students to lead their lives with personal integrity, professional ethics and civic responsibility in a global society.
- To prepare next generation of skilled professionals to successfully compete in the diverse global market.
- To promote campus environment that welcomes and honors women and men of all races, creeds and cultures, values and intellectual curiosity, pursuit of knowledge and academic integrity and freedom.
- To offer wide variety of off-campus education and training programmes to individuals and groups.
- To stimulate collaborative efforts with Industry, Universities, Government and Professional Societies.
- To facilitate public understanding of technical issues and achieve excellence in the operations of the institute.

Quality Policy

- Our Organization delights customers (Student, Parents and Society) by providing value added quality education to meet the National and International requirements.
- We also provide necessary steps to train the students for placement and continue to improve our methods of education to the students through effective. Quality Management System, Quality Policy and Quality Objectives.

Aditya College of Engineering and Technology

Approved by AICTE - New Delhi, Affiliated to VTU - Belagavi

2/2, 2/8, 2/9, Kamakshipura, Hesaraghatta Hobli, Yelahanka, Bengaluru, Karnataka - 560 089

Department of Computer Science & Engineering

Department Vision

To deliver high quality education anbd research in electronics and communication engineering, foster industry collaboration anbd cultivate enginheers who are ethical and principles, advancing connectivity and contributing to societal advancement.

Department Mission

To foster a dynamic state of the art learning Environment that combines advanced research and industry partnerships, nurturing electronics and communication engineers contributing to technological advancements.

To improve problem solving and support continuous learning that will enlighten the students to pursue higher studies and advance the career in research.

To develop engineers with robust communication skills and the ability to lead ethical values, addressing the social needs effectively.

Aditya College of Engineering and Technology
Approved by AICTE - New Delhi, Affiliated to VTU - Belagavi
2/2, 2/8, 2/9, Kamakshipura, Hesaraghatta Hobli, Yelahanka, Bengaluru, Karnataka - 560 089

PROGRAM EDUCATIONAL OBJECTIVES (PEO's)

After four Years of Graduation, our graduates are able to:

- Describe the structures of R Programming.
- Illustrate the basics of Data Preparation with real world examples.
- Apply the Graphical Packages of R for visualization.
- Apply various Statistical Analysis methods for data analytic.

PROGRAM SPECIFIC OUTCOMES (PSO'S)

At the end of the B.E Department of Computer Science & Engineering program, students are expected to have developed the following program specific outcomes.

PSO1: Build Analog and Digital Electronic systems for Multimedia Applications, VLSI and Embedded Systems in Interdisciplinary Research / Development.

PSO2: Design and Develop Communication Systems as per Real Time Applications and Current Trends.

COURSE OBJECTIVES

1. To design and implement various algorithms in C/C++ programming using suitable development tools to address different computational challenges.

2. To apply diverse design strategies for effective problem-solving.

3. To Measure and compare the performance of different algorithms to determine their efficiency and suitability for specific tasks.

COURSE OUTCOMES

1. Develop programs to solve computational problems using suitable algorithm design strategy.
2. Compare algorithm design strategies by developing equivalent programs and observing running times for analysis (Empirical).
3. Make use of suitable integrated development tools to develop programs
4. Choose appropriate algorithm design techniques to develop solution to the computational and complex problems.
5. Demonstrate and present the development of program, its execution and running time(s) and record the results/inferences.s.

Aditya College of Engineering and Technology
Approved by AICTE - New Delhi, Affiliated to VTU - Belagavi
2/2, 2/8, 2/9, Kamakshipura, Hesaraghatta Hobli, Yelahanka, Bengaluru, Karnataka - 560 089

PROGRAM OUTCOMES

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization for the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs.
4. **Conduct investigations of complex problems:** An ability to design and conduct scientific and engineering experiments, as well as to analyze and interpret data to provide valid conclusions
5. **Modern tool usage:** Ability to apply appropriate techniques, modern engineering and IT tools, to engineering problems.
6. **The engineer and society and Environment and sustainability:** An ability to apply reasoning to assess societal, safety, health and cultural issues and the consequent responsibilities relevant to the professional engineering practice and An ability to understand the impact of professional engineering solutions in societal and environmental contexts.
7. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
8. **Individual and team work:** Ability to function effectively as an individual, and as a member or leader in a team, and in multidisciplinary tasks.
9. **Communication:** Ability to communicate effectively on engineering activities with the engineering community such as, being able to comprehend and write effective reports and design documentation, make effective presentations.
10. **Project management and finance:** An ability to apply knowledge, skills, tools, and techniques to project activities to meet the project requirements with the aim of managing project resources properly and achieving the project's objectives.
11. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Aditya College of Engineering and Technology
Approved by AICTE - New Delhi, Affiliated to VTU - Belagavi
2/2, 2/8, 2/9, Kamakshipura, Hesaraghatta Hobli, Yelahanka, Bengaluru, Karnataka - 560 089

Department of Computer Science & Engineering

SYLLABUS

ANALYSIS AND DESIGN OF ALGORITHMS LAB

Subject Code : BCSL404 IA Marks : 50

No. of Practical Hrs/Week : 03 Exam Hours : 03

Total no. of Practical Hrs.: 42 Exam Marks : 50

List of Experiments

1. Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm.

2. Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Prim's algorithm.

3. a. Design and implement C/C++ Program to solve All-Pairs Shortest Paths problem using Floyd's algorithm.
   b. Design and implement C/C++ Program to find the transitive closure using Warshal's algorithm.

4. Design and implement C/C++ Program to find shortest paths from a given vertex in a Weighted connected graph to other vertices using Dijkstra's algorithm.

5. Design and implement C/C++ Program to obtain the Topological ordering of vertices in a given digraph.

6, Design and implement C/C++ Program to solve 0/1 Knapsack problem using Dynamic Programming method.

7 . Design and implement C/C++ Program to solve discrete Knapsack and continuous Knapsack problems using greedy approximation method.

8. Design and implement C/C++ Program to find a subset of a given set S = {sl , s2,.....,sn} of n positive integers whose sum is equal to a given positive integer d.

9. Design and implement C/C++ Program to sort a given set of n integer elements using Selection Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

10. Design and implement C/C++ Program to sort a given set of n integer elements using Quick Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

11. Design and implement C/C++ Program to sort a given set of n integer elements using Merge Sort method and compute its time complexity. Run the program for varied values of n> 5000, and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

12. Design and implement C/C++ Program for N Queen's problem using Backtracking.

Suggested Learning Resources:

1. Introduction to the Design and Analysis of Algorithms, By Anany Levitin, 3rd Edition (Indian), 2017, Pearson.

Reference books

1. Computer Algorithms/C++, Ellis Horowitz, SatrajSahni and Rajasekaran, 2nd Edition, 2014, Universities Press.
2. Introduction to Algorithms, Thomas H. Cormen, Charles E. Leiserson, Ronal L. Rivest, Clifford Stein, 3rd Edition, PHI.
3. Design and Analysis of Algorithms, S. Sridhar, Oxford (Higher Education)

COURSE ASSESSMENT AND EVALUATION						
Direct Assessment Methods						
What		**To whom**	**When/Where (Frequency inthe course)**	**Max. Marks**	**Evidence Collected**	**Contributing to Course outcomes**
CIE	Record & Observation	Students	Every lab session (Avg. of all experiment marks)	25	Observation book written at each lab + Record submitted at each lab + Viva	CO1 – CO4
	IA Test		One	25	Blue Books	CO1 – CO4

MAPPING OF COURSE OUTCOMES WITH PROGRAMOUTCOMES															
		PO												**PSO**	
		1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**1**	**2**
CO	**1**	3	3	1	2	1									
	2	3	2	3	2	2									
	3	2	2	3	3	1									
	4	2	3	2	2	1									

	5	2	2	3	3	3								
	Sum	12	1 2	12	12	8								
	Avg	2	2	2	2	1								

3: High correlation, 2: Medium correlation, 1: Low correlation

INSTRUCTIONS TO THE CANDIDATES

- Student should come with thorough preparation for the experiment to be conducted.
- Student should take prior permission from the concerned faculty before availing the leave.
- Student should come with proper dress code and to be present on time in the laboratory.
- Student will not be permitted to attend the laboratory unless they bring the practical record fully completed in all respects pertaining to the experiment conducted in the previous class.
- Student will not be permitted to attend the laboratory unless they bring the observation book fully completed in all respects pertaining to the experiment to be conducted in present class.
- Experiment should be started conducting only after the staff-in-charge has checked the circuit diagram.
- All the calculations should be made in the observation book. Specimen calculations for one set of readings have to be shown in the practical record.
- Wherever graphs to be drawn, A-4 size graphs only should be used and the same should be firmly attached in the practical record.
- Practical record and observation book should be neatly maintained.
- Student should obtain the signature of the staff-in-charge in the observation book after completing each experiment.
- Theory related to each experiment should be written in the practical record before procedure in your own words with appropriate references.

CONTENTS

SL. No.	Name of the Experiment	Page No
	First cycle (Hardware Experiments)	
1	Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm.	16-19
2	Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Prim's algorithm.	20-25
3	a. Design and implement C/C++ Program to solve All-Pairs Shortest Paths problem using Floyd's algorithm. b. Design and implement C/C++ Program to find the transitive closure using Warshal's algorithm.	26-29
4	Design and implement C/C++ Program to find shortest paths from a given vertex in a weighted connected graph to other vertices using Dijkstra's algorithm.	30-34
5	Design and implement C/C++ Program to obtain the Topological ordering of vertices in a given digraph.	35-37
6	Design and implement C/C++ Program to solve 0/1 Knapsack problem using Dynamic Programming method.	38-40
7.	Design and implement C/C++ Program to solve discrete Knapsack and continuous Knapsack problems using greedy approximation method.	41-42
8.	Design and implement C/C++ Program to find a subset of a given set S = {sl , s2,.....,sn} of n positive integers whose sum is equal to a given positive integer d.	43-44
9.	.Design and implement C/C++ Program to sort a given set of n integer elements using Selection Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.	45-47
10.	Design and implement C/C++ Program to sort a given set of n integer elements using Quick Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.	48-51
11.	Design and implement C/C++ Program to sort a given set of n integer elements using Merge Sort method and compute its time complexity. Run the program for varied values of n> 5000, and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.	52-55
12	Design and implement C/C++ Program for N Queen's problem using Backtracking.	56-58
13	Algorithms Laboratory Viva Questions	59-65

Sl No.	Name of the Experiment	Date			Observation Marks (max. 20)	Record Marks (max. 10)	Signature(Student)	Signature(Faculty)
		Conduction	Repetition	Submission of Record				
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
Average								

Introduction

Analysis and Design of Algorithms (ADA) is a core field of computer science that focuses on creating efficient procedures to solve computational problems and evaluating their performance.

1. Fundamental Concepts

- **Algorithm**: A finite, well-defined sequence of instructions designed to solve a problem or perform a task.
- **Design**: The process of selecting mathematical models and data structures to create a step-by-step procedure for a problem.
- **Analysis**: The theoretical evaluation of an algorithm's resource requirements (time and memory) as input size increases.

2. Characteristics of a Good Algorithm

For a procedure to be considered an algorithm, it must satisfy several criteria:

- **Finiteness**: It must terminate after a finite number of steps.
- **Definiteness**: Each step must be clear and unambiguous.
- **Input/Output**: It must take zero or more inputs and produce at least one output.
- **Effectiveness**: Every instruction must be basic enough to be carried out in a finite amount of time.
- **Language Independence**: It should be representable in pseudocode or flowcharts, independent of any specific programming language.

3. Performance Analysis Metrics

Efficiency is primarily measured through two complexities:

- **Time Complexity**: The amount of computer time required for an algorithm to run to completion, often expressed using Asymptotic Notation like Big-O (), Omega (), and Theta ().
- **Space Complexity**: The amount of memory or storage required during execution.
- **Analysis Cases**:

Worst-case: Maximum resources needed for any input of size.

Best-case: Minimum resources needed.

Average-case: Expected resources needed over all possible inputs

4. Common Design Strategies

Modern algorithm design relies on several standard techniques:

- **Divide and Conquer**: Breaking a problem into smaller sub-problems, solving them recursively, and combining the results (e.g., Merge Sort, Quick Sort).
- **Greedy Method**: Making the locally optimal choice at each step with the hope of finding a global optimum (e.g., Prim's algorithm, Kruskal's algorithm).
- **Dynamic Programming**: Storing results of sub-problems to avoid redundant calculations (e.g., Fibonacci sequence, Knapsack problem).
- **Backtracking**: Building a solution incrementally and removing candidates that fail to satisfy constraints (e.g., N-Queens problem).

5. Tools for Learning ADA

To master this subject, students often use the following resources:

- **Standard Textbooks**: *"Introduction to Algorithms"* by Cormen et al. is widely considered the primary reference.
- **Online Courses**: Platforms like MIT OpenCourseWare provide comprehensive intermediate algorithm materials.
- **Practice Platforms**: Use the GeeksforGeeks DAA Tutorial or TutorialsPoint for step-by-step conceptual guides.

Experiment No. 1 **Date:24/02/ 2026___**

1.Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm.

Algorithm

Kruskal(n,m,E)
//Inputs:
n – Number of vertices in the graph
m – Number of edges in the graph
E – edge list consisting of set of edges along with equivalent weights

//Outputs:
Count ← 0
k ←0
sum ←0

// Create a forest with n vertices
for i ←0 to n do
parent[i] ← i
end for

while(count≠n-1 and E≠Φ)
Select an edge (u,v) with least cost

i ← find(u,parent)
j ← find(v,parent)

if(i!=j)
t[k][0] ←u;
t[k][1] ←v;
k++;
count++;
sum ←sum +cost(u,v);
union(i,j,parent);
end if
if(count!= n-1)
write("Spanning tree does not exists\n");
return
end if
write("Spanning Tree is shown below\n);
for i ← 0 to n-2 do
write(t[i][0],t[i][1]);
write("Cost of spanning tree",sum);

/*Program*/

```
# include <stdio.h>
int a,b,i,j,u,v,n;
int visited[10],edge=1, min, mincost=0, cost[10][10];

void kruskal(int n, int cost[10][10])
{
        printf("The edge of the spanning tree are \n");
        while(edge<n)
        {
                for(i=1,min=999;i<=n;i++)
                for(j=1;j<=n;j++)
                if(cost[i][j]<min)
                {
                        min=cost[i][j];
                        a=u=i;
                        b=v=j;
                }
                while(visited[u])
                u=visited[u];
                while(visited[v])
                v=visited[v];
                if(u!=v)
                {
                        edge++;
                        printf("\n Edge (%d,%d)=%d", a,b,min);
                        mincost=mincost+min;
                        visited[v]=u;
                }
                cost[a][b]=cost[b][a]=999;
        }
printf("\n The minimum cost =%d", mincost);
}

int main( )
{
        printf("Enter the number of vertices:\n");
        scanf("%d", &n);
        printf("Enter the cost adjacency matrix:\n");
        for(i=1;i<=n;i++)
        for(j=1;j<=n;j++)
        scanf("%d", &cost[i][j]);
        kruskal(n,cost);
        return 0;
}
```

OUTPUT:

Enter the number of vertices:

6

Enter the cost adjacency matrix:

0	3	999	999	6	5
3	0	1	999	999	4
999	1	0	6	999	4
999	999	6	0	8	5
6	999	999	8	0	2
5	4	4	5	2	0

The edge of the spanning tree are

Edge (2,3)=1
Edge (5,6)=2
Edge (1,2)=3
Edge (2,6)=4
Edge (4,6)=5

The minimum cost =15

Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

Solution 1:

Enter how many nodes: 6

Enter the cost adjacency matrix:

	0	1	2	3	4	5
0	0	60	10	9999	9999	9999
1	60	0	9999	20	40	70
2	10	9999	0	9999	9999	50
3	9999	20	9999	0	9999	80
4	9999	40	9999	9999	0	30
5	9999	70	50	80	30	0

Spanning tree exists and the spanning tree is

0,2
1,3
4,5
1,4
2,5

The cost of the spanning tree = 150

Solution 2:

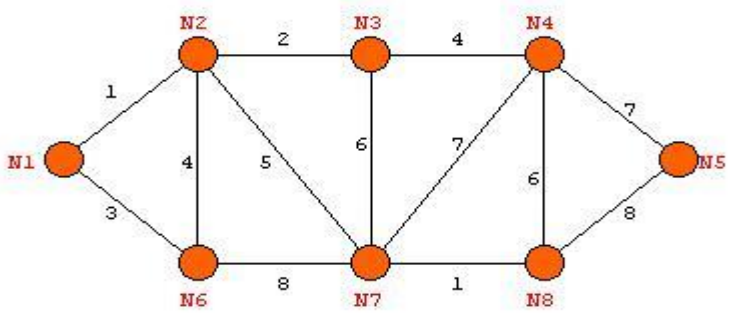
Enter how many nodes: 3

Enter the cost adjacency matrix:

0	20	9999
20	0	9999
9999	9999	0

Spanning tree does not exist.

Tracing of Kruskal's Algorithm



N1 to N2 - cost is 1 - add to tree
N7 to N8 - cost is 1 - add to tree
N2 to N3 - cost is 2 - add to tree
N1 to N6 - cost is 3 - add to tree
N2 to N6 - cost is 4 - reject because it forms a circuit
N3 to N4 - cost is 4 - add to tree
N2 to N7 - cost is 5 - add to tree
N3 to N7 - cost is 6 - reject because it forms a circuit
N4 to N8 - cost is 6 - reject because it forms a circuit
N4 to N7 - cost is 7 - reject because it forms a circuit
N4 to N5 - cost is 7 - add to tree

Stop here, because n -1 edges have been added.
The minimum spanning tree, with a total weight of 23 is obtained.

Experiment No. 2 **Date: 03 /03/ 2026__**

2. Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Prim's algorithm.

ALGORITHM PRIM (n,w)

```
Input:          n- Number of vertices
                w-Cost adjacency matrix with values>=0
Output: Spanning tree if exists

/* Find the source vertex from which an edge with least cost starts */
        min←9999;
        source←0;
        for i←0 to n-1 do
                for j←0 to n-1 do
                        if(a[i][j]!=0 && a[i][j]<=min)
                                min←a[i][j];
                                source←i;
                end if
        end for
end for

/* Initialization to find minimum spanning tree */
for i←0 to n-1 do
        d[i] ←a[source][i];
        s[i] ←0,p[i] ←source;
end for
s[source] ←1;   /*Add source to S */
        sum←0;                  /* Initial cost of minimum spanning tree */
        k←0;            /* used as index to store the edges selected */
        for i←0 to n-1 do
                 Find u and d[u] such that d[u] is minimum and u in V-S
                Add u to S
                Select an edge with least cost
                Add cost to selected edge to get total cost of minimum spanning tree

                for every v belongs to V-S do
                        if( a[u][v]<d[v])
                                d[v]←a[u][v];
                                p[v]←u;
end if
end for
                        /* Check for the spanning tree */
if(sum>=9999)
                write "Spanning tree does not exist";
        else
                Write "Spanning tree exists and minimum spanning tree is";
end if
```

/*Program*/

```
# include<stdio.h>
int n,a[10][10];
void min_spa_tree()
{
        int i,j,u,v,min;
        int sum;
        int k;
        int t[10][2];
        int p[10];
        int d[10];
        int s[10];
        int source;

        min=9999;
        source=0;
        for(i=0;i<n;i++)
        {
                for(j=0;j<n;j++)
                {
                        if(a[i][j]!=0 && a[i][j]<=min)
                        {
                                min=a[i][j];
                                source=i;
                        }
                }
        }

        for(i=0;i<n;i++)
        {
                d[i]=a[source][i];
                s[i]=0;
                p[i]=source;
        }
        s[source]=1;
        sum=0;
        k=0;

        for(i=1;i<n;i++)
        {
                min=9999;
                u=-1;
                for(j=0;j<n;j++)
                {
                        if(s[j]==0)
                        {
                                if(d[j]<=min)
                                {
                                        min=d[j];
```

```
                                        u=j;
                                }
                        }
                }
                t[k][0]=u;
                t[k][1]=p[u];
                k++;
                sum+=a[u][p[u]];
                s[u]=1;
                for(v=0;v<n;v++)
                {
                        if(s[v]==0 && a[u][v]<d[v])
                        {
                                d[v]=a[u][v];
                                p[v]=u;
                        }
                }
        }

        if(sum>=9999)
        printf("spanning tree does not exits\n");
        else
        {
                printf("spanning tree exists and min spanning tree is \n");
                for(i=0;i<n-1;i++)
                {
                        printf("%d", t[i][0]);
                        printf(" ");
                        printf("%d", t[i][1]);
                        printf("\n");
                }
                printf("The cost of the spanning tree is %d", sum);
                printf("\n");
        }
}

void read_data( )
{
        int i,j;
        printf("Enter the no of nodes\n");
        scanf("%d", &n);
        printf("Enter the cost adjacency matrix\n");
        for(i=0;i<n;i++)
        {
                for(j=0;j<n;j++)
                {
                        scanf("%d", &a[i][j]);
                }
        }
}

int main( )
{
```

```
        void read_data( );
        void min_spa_tree( );
        read_data( );
        min_spa_tree( );
        return 0;
}
```

OUTPUT:

```
Enter the no of nodes
6
Enter the cost adjacency matrix

0       3     9999   9999   6       5
3       0      1     9999   9999    4
9999    1      0      6     9999    4
9999   9999    6      0      8      5
6      9999   9999    6      0      2
5       4      4      5      2      0
spanning tree exists and min spanning tree is
1  2
0  1
5  2
4  5
3  5
The cost of the spanning tree is 15
```

Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

Solution 1:

```
Enter the nos of vertices:
6

Enter cost adjacency matrix:
(Enter 9999 to indicate no edge)
0       60      10      9999    9999    9999
60      0       9999    20      40      70
10      9999    0       9999    9999    50
9999    20      9999    0       9999    80
9999    40      9999    9999    0       30
9999    70      50      80      30      0

Spanning tree exists and the spanning tree is
2→0
5→2
4→5
1→4
3→1

The cost of the spanning tree = 150
```

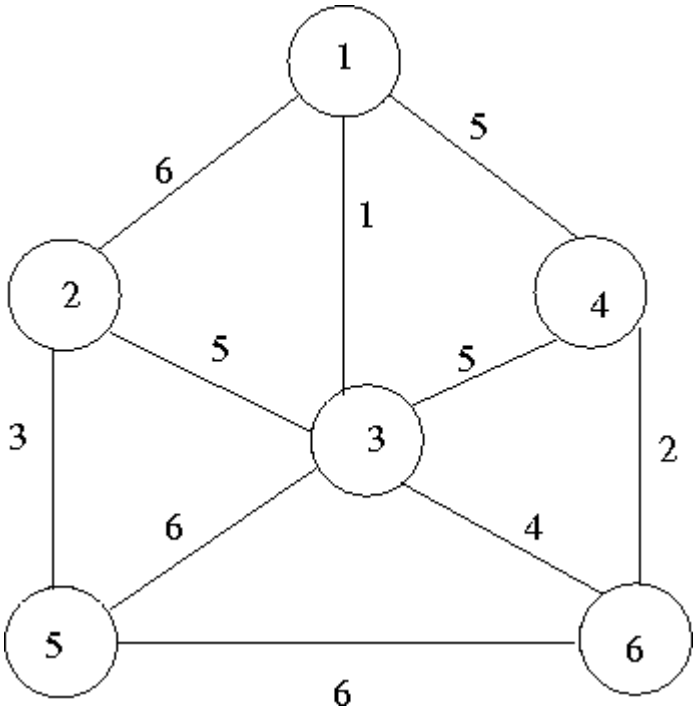
Solution 2:

Enter the nos of vertices:
3

Enter cost adjacency matrix:
(Enter 9999 to indicate no edge)
0 20 9999
20 0 9999
9999 9999 0

Spanning tree does not exist.

Figure 8.12: An example graph for illustrating Prim's algorithm



- At each step, we can scan lowcost to find the vertex in V - U that is closest to U. Then we update lowcost and closest taking into account the new addition to U.
- Complexity: $O(n^2)$

Example: Consider the digraph shown in Figure 8.12.

Step 1

$S = \{1\}$ $V - S = \{2, 3, 4, 5, 6\}$

closest $V - S$	S	lowcost
2	1	6
3	1	1
4	1	5
5	1	∞
6	1	∞

Select vertex 3 to include in ***U***

Step 2

$S = \{1, 3\}$ $V - S = \{2, 4, 5, 6\}$

closest $V - U$	U	lowcost
2	3	5
4	1	5
5	3	6
6	3	4

Now select vertex 6

Step 3

$S = \{1, 3, 6\}$ $V - S = \{2, 4, 5, 6\}$

closest $V - U$	U	lowcost
2	3	5
4	6	2
5	3	6

Now select vertex 4, and so on

Experiment No. 3 **Date: 10_/03_/ 2026__**

3.

A. Design and implement C/C++ Program to solve All-Pairs Shortest Paths problem using Floyd's algorithm.

B. Design and implement C/C++ Program to find the transitive closure using Warshal's algorithm.

(a). Implement Floyd's algorithm for the All-Pairs- Shortest-Paths problem.

ALGORITHM FLOYD(W[1..n, 1..n])

ALGORITHM *Floyd*(W[1..n, 1..n])
//Implements Floyd's algorithm for the all pairs shortest paths problem
//Input: The weight matrix W of a graph
//Output: The distance matrix of the shortest paths' lengths
D ← W //is not necessary if W can be overwritten
for k ← 1 **to** n **do**
  for i ← 1 **to** n **do**
    for j ← 1 **to** n **do**
      D[i, j] ← min{D[i, j], D[i, k] + D[k,j]}
return D

/*PROGRAM */

```
#include<stdio.h>

void main()
{
  int a[10][10],i,j,k,n;
  printf("enter the no.of nodes\n");
  scanf("%d",&n);
  printf("enter the cost adjacency matrix,'1000' for no direct path\n");
  for(i=1;i<=n;i++)
  {
  for(j=1;j<=n;j++)
  scanf("%d",&a[i][j]);
  a[i][j]=0;
  }
  for(k=1;k<=n;k++)
  for(i=1;i<=n;i++)
  for(j=1;j<=n;j++)
  if((a[i][k]+a[k][j])<a[i][j])
  a[i][j]=a[i][k]+a[k][j];
  printf("output path matrix\n");
  for(i=1;i<=n;i++)
  {
   for(j=1;j<=n;j++)
   printf("%6d",a[i][j]);
   printf("\n");
```

```
  }
}

/*Output
enter the no.of nodes
4
enter the cost adjacency matrix,'1000' for no direct path
0    1000  3      1000
2       0   1000  1000
1000   7    0       1
6      1000 1000  0
output path matrix
    0   10    3    4
    2    0    5    6
    7    7    0    1
    6   16    9    0  */
```

(b). Implement Warshall's Algorithm **f** ind the transitive closure.

// Input: Adjacency matrix a of size nxn
//Output:Path matrix of size nxn

```
for k←0 to n-1 do
        for i←0 to n-1 do
                for j←0 to n-1 do
                if(a[i,k]=1 and a[k,j]=1) then
                        a[i][j]=1
                endif
                endfor
        endfor
endfor
return
```

/*PROGRAM*/

```
#include<stdio.h>
int n;
int a[10][10];
int p[10][10];

void write_data( )
{
        int i,j;
        printf("The path matrix is shown below\n");
        for(i=0;i<n;i++)
        {
                for(j=0;j<n;j++)
                {
                        printf("%d", p[i][j]);
                        printf(" ");
                }
```

```
                printf("\n");
        }

}

void read_data( )
{
        int i,j;
        printf("Enter the no of nodes\n");
        scanf("%d", &n);
        printf("Enter the adjacency matrix\n");
        for(i=0;i<n;i++)
        {
                for(j=0;j<n;j++)
                {
                        scanf("%d", &a[i][j]);
                }
        }
}

void path_matrix( )
{
        int i,j,k;
        for(i=0;i<n;i++)
        {
                for(j=0;j<n;j++)
                p[i][j]=a[i][j];
        }



        for(k=0;k<n;k++)
        {
                for(i=0;i<n;i++)
                {
                        for(j=0;j<n;j++)
                        {
                                if((p[i][k]==1 && p[k][j]==1))
                                p[i][j]=1;
                        }
                }
        }
}

int main( )
{
        read_data();
        path_matrix();
        write_data();
}
```

OUTPUT:

```
Enter the no of nodes
4
Enter the adjacency matrix
0  1  0  0
0  0  0  1
0  0  0  0
1  0  1  0
The path matrix is shown below
1  1  1  1
1  1  1  1
0  0  0  0
1  1  1  1
```

Experiment No. 4 **Date:17//03/2026 __**

4. Design and implement C/C++ Program to find shortest paths from a given vertex in a weighted connected graph to other vertices using Dijkstra's algorithm.

ALGORITHM: DIJKSTRA ()

```
//Purpose: To compute the shortest distance and shortest path from given source to destination
//Input: n-number of vertices in graph
//        Cost[][]-Cost adjacency matrix with values >=0
//        Source-Source vertex
//        dest-destination vertex

//output: d-shortest distance from source to all other nodes
//          p-shortest distance from source to destination
//          s-gives the nodes that are visited and not visited

for i=0 to n-1 do
      d[i]=cost[source,i]
      p[i]=source
      selected[i]=0
end for

selected[source]=1 //add source to selected
for i=0 to n-1 do
       find u and d[u] such that d[u] is minimum and u ϵ V-s
       add u to selected
       if(u=destination)break;
       for every v ϵ V-selected do       //for v=0 to n-1
                if(d[u]+w[u,v]<d[v])
                          d[v]=d[u]+w[u,v]
                          p[v]=u
                end if
       end for
end for
```

/*PROGRAM */

```
#include<stdio.h>
void dijstra(int n, int v, int cost[10][10], int d[])
{
	int count,u,i,j,s[10],min;
	for(i=1;i<=n;i++)
	{
		s[i]=0;
		d[i]=cost[v][i];
	}
	s[v]=1;
	d[v]=1;
	count=2;
	while(count<=n)
	{
		min=999;
		for(j=1;j<=n;j++)
		if(d[j]<min && s[j]==0)
		{
			min=d[j];
			u=j;
		}
		s[u]=1;
		count++;
		for(j=1;j<=n;j++)
		if((d[u]+cost[u][j]<d[j])&&s[j]==0)
		d[j]=d[u]+cost[u][j];
	}
}
int main( )
{
	int n,i,j,v,c[10][10],d[10];
	printf("Enter the no of nodes");
	scanf("%d", &n);
	printf("Enter the cost adjacency matrix\n");
	for(i=1;i<=n;i++)
	for(j=1;j<=n;j++)
	scanf("%d", &c[i][j]);
	printf("Enter the source vertex");
	scanf("%d", &v);
	dijstra(n,v,c,d);
	printf("The shortest path");
	for(j=1;j<=n;j++)
	if(j!=v)
	printf("\n%d->%d=%d", v,j,d[j]);
	return 0;
}
```

OUTPUT1:

```
Enter the no of nodes
3
Enter the cost adjacency matrix
10   5  6
4    2  3
1    8  9
Enter the source vertex
2
The shortest path
2->1=4
2->3=3
```

OUTPUT2:

```
Enter the no of nodes
6
Enter the cost adjacency matrix
0      3      999    999    6      5
3      0      1      999    999    4
999    6      0      6      999    4
999    999    6      0      8      5
6      999    999    8      0      2
5      4      4      5      2      0
Enter the source vertex
2
The shortest path
2->1=3
2->3=1
2->4=7
2->5=6
2->6=4
```

OUTPUT3:

Enter how many node: 6
Enter the starting vertex:6

Enter the cost matrix:

0	15	10	999	45	999
999	0	15	999	20	999
20	999	0	20	999	0
999	10	999	0	35	999
999	999	999	30	0	999
999	999	999	4	999	0

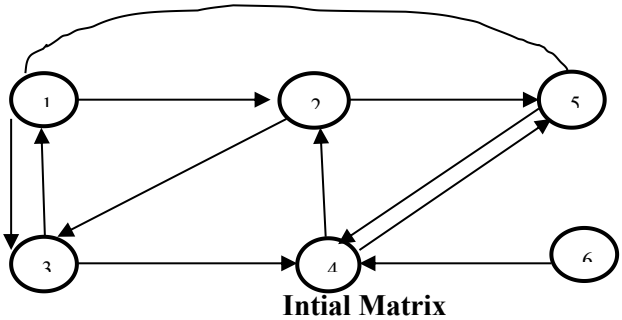
The given graph in matrix format:

0	15	10	999	45	999
999	0	15	999	20	999
20	999	0	20	999	0
999	10	999	0	35	999
999	999	999	30	0	999
999	999	999	4	999	0

Single vertex shortest paths:

From node 6 to node 1: 49
From node 6 to node 2: 14
From node 6 to node 3: 29
From node 6 to node 4: 4
From node 6 to node 5: 34
From node 6 to node 6: 0

Tracing of DIJKSTAR'S ALGORITHM:-



Cost **Intial Matrix**

	1	2	3	4	5	
1	0	15	10	999	45	999
2	999	0	15	999	20	999
3	20	999	0	20	999	0
4	999	10	999	0	35	999
5	999	999	999	30	0	999
6	999	999	999	4	999	0

	A	B	C	D
d[1]	999	999	999	49
d[2]	999	14	14	14
d[3]	999	999	29	29
d[4]	4	4	4	4

d[5]	999	39	34	34
d[6]	0	0	0	0

Tracing Table:-

s	**v=V-s**	**d[v]=min(d(v),d[u]+w(u,v))**	**d[u]—>min**
6	1,2,3,4,5		4,4
6,4	1,2,3,5	d[0]=min(999,4+999)=999 d[1]=min(999,4+10)=14 d[2]=min(999,4+999)=999 d[4]=min(999,4+35)=39	2,14
6,4,2	1,3,5	d[0]=min(999,14+999)=999 d[2]=min(999,14+15)=29 d[4]=min(39,14+20)=34	3,29
6,4,2,3	1,5	d[0]=min(999,29+20)=49 d[4]=min(34,29+999)=34	5,34
6,4,2,3,5	1	d[0]=min(49,34+999)=49	1,49

Experiment No. 5 **Date: 24_/03/ 2026__**

5.Design and implement C/C++ Program to obtain the Topological ordering of vertices in a given digraph.

Topological ordering

```
ALGORITHM: Topological_order(a,n,s)
//Purpose: To obtain the topological ordering of vertices in a given digraph.
//Input: a- adjacency matrix of the given graph.
         n- the number of vertices in the graph.
// Output: s-indicates the jobs that are to be executed in the order.
for j ← 0 to n-1 do // obtain indegree of each vertex
      sum ← 0
      for i ← 0 to n-1 do
      sum ← sum+a[i]
      end for
top ← -1 // Place independent jobs on the stack
for i ← 0 to n-1 do
       if(indegree[i]=0)
         top← top+1
         s[top] ← i
     end for
     while top ≠ 1 // Find the topological sequence
u ← s[top] // Delete the job from the stack
top ← top-1
Add u to the solution vector T
For each vertex v adjacent to u
Decrement indegree[v] by one
If(indegree[v]=0)
Top ← top+1
S[top]←v
End if
End for
End while
Output t
Return
```

/***PROGRAM***/

```
# include < stdio.h>

/* To read the adjacency matrix */
void read_adjacency_matrix(int n, int a[10][10])
{
        int i, j;

        for (i = 0; i < n; i++)
        {
                for (j = 0; j < n; j++)
                {
                        scanf("%d", &a[i][j]);
                }
```

```
        }
}

/*find the indegree of all vertices*/
void find_indegre(int n, int a[10][10], int indegre[])
{
        int i, j, sum;

        for (j = 0; j < n; j++)
        {
                sum = 0;

                for (i = 0; i < n; i++)                 // add the column values
                {
                        sum += a[i][j];
                }

                indegre[j] = sum;
        }
}

/* function to find solution (Topological sort) */
void topological_sort (int n, int a[10][10])
{
        int i,  k, u, v, top, t[10], indegre[10], s[10];

        find_indegre(n, a, indegre);
                top = -1;
        k = 0;

        for (i = 0; i < n; i++)
        {
                if (indegre[i] == 0)
                {
                        s [++top] = i;
                }
        }

        while (top != -1)
        {
                u = s[top--];
                t[k++] = u;

                for (v = 0; v < n; v++)
                {
                        if (a[u][v] == 1)
                        {
                                indegre[v]--;
                                if (indegre[v] == 0)
                                {
                                        s[++top] = v;
                                }
                        }
```

```
                }
        }

        printf ("The topological sequence");
        for (i = 0; i < n; i++)
                printf("%d ", t[i]);
}

void main ( )
{
        int n, a[10][10];

        printf("Enter the number of jobs\n");
        scanf("%d", &n);

        printf("Enter the adjacency matrix");
        read_adjacency_matrix(n,a);
        topological_sort(n,a);
}
```

Output

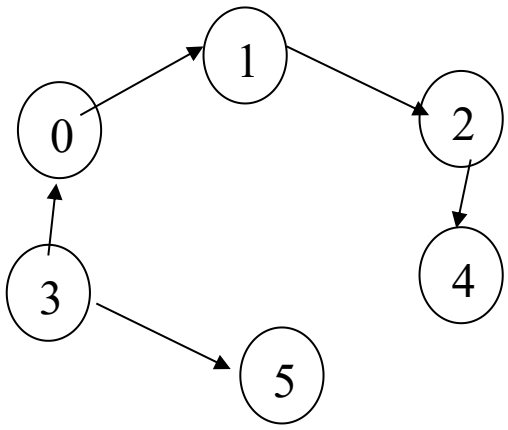
Topological ordering for a digraph

Enter the number of vertices: 6

Enter the adjacency matrix:

0	1	0	0	0	0
0	0	1	0	0	0
0	0	0	0	1	0
1	0	0	0	0	1
0	0	0	0	0	0
0	0	0	0	0	0

GRAPH



STACK	U=POP()	SOLUTION (T)	V=ADJ(V)	INDEGREE					
				[0]	[1]	[2]	[3]	[4]	[5]
				1	1	1	0	1	1
3	3	3	0,5	0	1	1	0	1	0
0,5	5	3,5	-	0	1	1	0	1	0
0	0	3,5,0	1	0	0	1	0	1	0
1	1	3,5,0,1	2	0	0	0	0	1	0
2	2	3,5,0,1,2	4	0	0	0	0	0	0

The topological order is: 3 → 5 → 0 → 1 → 2 → 4

Experiment No. 6 **Date: 31/03/2026 ___**

6. Design and implement C/C++ Program to solve 0/1 Knapsack problem using Dynamic Programming method.

Algorithm:
Knapsack(n,m,w,p,v)
//Inputs:
n – Number of objects to be selected
m – Capacity of the knapsack
w – weights of all the objects
p – profits of all the objects

//Outputs:
v – the optimal solution for the number of objects selected with specified remaining capacity

```
for ← 0 to n do
    for ← 0 to m do
        if(i==0 or j==0)
            v[i][j]=0;
     else if(j<w[i])
             v[i][j]=v[i-1][j];
    else
            v[i][j]=max(v[i-1,j],v[i-1][j-w[i]]+p[i]);
            endif
    endfor
endfor

return
```

/***PROGRAM***/

```
#include<stdio.h>
#include<stdlib.h>

int weights[10], profits[10], v[10][10];
int Max(int a,int b)
{
        if(a>b)
        return a;
        else
        return b;
}
int knapsack(int n,int w)
{
        int i,j;
        for(i=0;i<=n;i++)
        {
                for(j=0;j<=w;j++)
                {
                        if(i==0||j==0)
                        v[i][j]=0;
                        else if(j-weights[i]>=0)
                        v[i][j]=Max(v[i-1][j],profits[i]+v[i-1][j-weights[i]]);
                        else
                        v[i][j]=v[i-1][j];
                }
        }
        return v[n][w];
}
void optimalsubset(int n,int w)
{
        int i,j;
        for(i=n,j=w;i>=1 && j>0;i--)
        {
                if(v[i][j]!=v[i-1][j])
                {
                        printf("item %d\n",i);
                        j=j-weights[i];
                }
        }
}
int main( )
{
        int n,w,mp,i,j;
        printf("enter the number of items \n");
        scanf("%d",&n);
        printf("enter the weights of each item\n");
        for(i=1;i<=n;i++)
```

```
        scanf("%d",&weights[i]);
        printf("enter the profit of each item\n");
        for(i=1;i<=n;i++)
        scanf("%d",&profits[i]);
        printf("enter the knapsack capacity\n");
        scanf("%d",&w);
        mp=knapsack(n,w);
        printf("solution of the knapsack \n");
        for(i=0;i<=n;i++)
        {
                for(j=0;j<=w;j++)
                printf("%d\t",v[i][j]);
                printf("\n");
        }
        printf("the maximal value is %d \n",mp);
        printf("the items of optimal subset are\n");
        optimalsubset(n,w);
        return 0;
}
```

OUTPUT:

```
enter the number of items
4
enter the weights of each item
2
1
3
2
enter the profit of each item
12
10
20
15
enter the knapsack capacity
5
solution of the knapsack
0       0       0       0       0       0
0       0       12      12      12      12
0       10      12      22      22      22
0       10      12      22      30      32
0       10      15      25      30      37

the maximal value is 37

the items of optimal subset are
item 4
item 2
item 1

The optimal solution is 37
X[0] X[1] X[2] X[3] X[4] =1 1 0 1
```

Experiment No. 7 **Date: 07/04/2026 ___**

In **0/1 knapsack**, items **cannot be divided**.
Using greedy approximation, we select items based on **highest profit/weight ratio**.

Algorithm: Discrete Knapsack (Greedy Approximation)

Start.

Input number of items n.

Input weight w[i] and profit p[i].

Input knapsack capacity W.

Compute ratio p[i] / w[i].

Sort items in descending order of ratio.

Initialize total_profit = 0.

For each item:

If w[i] <= W

Select the item.

total_profit += p[i]

W = W - w[i]

Display selected items and total profit.

Stop.

/* PROGRAM*/

Discrete (0/1) Knapsack (Greedy Approximation)

```
#include <stdio.h>
int main() {
    int n, i, j;
    float weight[20], profit[20], ratio[20];
```

```
    float capacity, temp, total_profit = 0;

    printf("Enter number of items: ");
    scanf("%d",&n);

    for(i=0;i<n;i++){
        printf("Enter weight and profit for item %d: ",i+1);
        scanf("%f%f",&weight[i],&profit[i]);
        ratio[i] = profit[i]/weight[i];
    }

    printf("Enter knapsack capacity: ");
    scanf("%f",&capacity);
    // Sorting
    for(i=0;i<n;i++){
        for(j=i+1;j<n;j++){
            if(ratio[i] < ratio[j]){
                temp = ratio[i];
                ratio[i] = ratio[j];
                ratio[j] = temp;

                temp = weight[i];
                weight[i] = weight[j];
                weight[j] = temp;
                temp = profit[i];
                profit[i] = profit[j];
                profit[j] = temp;
            }
        }
    }
    for(i=0;i<n;i++){
        if(weight[i] <= capacity){
            printf("Item %d selected\n",i+1);
            capacity -= weight[i];
            total_profit += profit[i];
        }
    }
 printf("Maximum profit = %.2f\n", total_profit);
return 0;
}
```

Experiment No. 8 **Date: 14_/04/2026 __**

7. Design and implement C/C++ Program to find a subset of a given set S = {sl , s2,.....,sn} of n positive integers whose sum is equal to a given positive integer d.

```
Algorithm SumofSub(cs, k, r)
// Input: Set of n numbers, w[1..n]
// Output: Solution vector, Boolean vector: X[1..n]
                                                    // Create left child (with kth element)
X[k] ← 1                                            //Try one branch of the tree
if (cs + w[k]) = d
for i ← 1 to k do
if X[i] = 1
print S[i]                                          //We have a solution
else
if cs+ w[k] + w[k + 1] <=    d
SumofSub (cs + w[k], k + 1, r – w[k])  //A solution is still possible, so
                        continue along this branch
if (cs + r – w[k] >=    d) and (cs + w[k + 1] <=    d)
X[k] ← 0
SumofSub(cs, k + 1, r – w[k])                       //Now try the other branch
end SumofSub.
```

/*PROGRAM*/

```
#include<stdio.h>
int  w[20], d,n,count,x[10], i,k;

void sumsubsets(int s, int k, int r)
{
        x[k]=1;
        if(s+w[k]==d)
        {
                printf("\n subset %d\t", ++count);
                for(i=0;i<=k;i++)
                if(x[i])
                printf("%d  ", w[i]);
        }
        else
        if(s+w[k]+w[k+1]<=d)
        sumsubsets(s+w[k],k+1,r-w[k]);
        if((s+r-w[k]>=d) && (s+w[k+1]<=d))
        {
                x[k]=0;
                sumsubsets(s,k+1,r-w[k]);
        }
}

int main( )
{
```

```
        int sum=0;
        printf("\n Enter the no. of elements");
        scanf("%d", &n);
        printf("Enter the elements :");
        for(i=0;i<n;i++)
        scanf("%d", &w[i]);
        printf("Enter the sum :\n");
        scanf("%d", &d);
        for(i=0;i<n;i++)
        x[i]=0;
        for(i=0;i<n;i++)
        sum+=w[i];
        if(sum<d || w[0]>d)
        printf("\n no subsets is possible");
        else
        sumsubsets(0,0,sum);
        return 0;
}
```

OUTPUT1:

```
Enter the no. of elements5
Enter the elements :1
2
3
4
5
Enter the sum :
6
 subset 1          1  2  3
 subset 2          1  5
 subset 3          2  4
```

OUTPUT2:

```
Enter the no. of elements2
Enter the elements :1
1
Enter the sum :
5
 no subsets is possible
```

OUTPUT3:

```
Enter number of elements: 5

Enter the set in increasing order
1 2 5 6 8

Enter the sum of subset d = 9

Solution is :  1      2      6
Solution is :   1      8
```

Experiment No. 9 **Date: 21_/04/2026 __**

9.Design and implement C/C++ Program to sort a given set of n integer elements using Selection Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

ALGORITHM SELECTIONSORT(A[0..n - 1])

ALGORITHM *SelectionSort*(A[0..n - 1])
//The algorithm sorts a given array by selection sort
//Input: An array A[0..n - 1] of orderable elements
//Output: Array A[0..n - 1] sorted in ascending order
for i ← 0 **to** n - 2 **do**
    min ← i
    for j ← i + 1 **to** n - 1 **do**
        if A[j] < A[min] min ← j
    swap A[i] and A[min]

/*PROGRAM*/

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>

void selectionsort(int *,int);
void swap(int [],int,int);

void main()
{
   int n,a[25000],i;
   char ans=0;
   clock_t start,stop;
   for(;;)
   {
   printf("enter the no. of elements\n");
   scanf("%d",&n);
//   printf("enter the array elements\n");
   for(i=0;i<n;i++)
//   scanf("%d",&a[i]);
   a[i]=rand();
   start=clock();
   selectionsort(a,n);
   stop=clock();
   printf("the sorted elements are:\n");
```

```
   for(i=0;i<n;i++)
   printf("%d\t",a[i]);
   printf("\nthe time taken for sorting is %f sec\n",(stop-start)/CLK_TCK);

   printf("\nlike to have another solution(y/n)?:");
                fflush(stdin);
                scanf("%c",&ans);
   if(ans!='y'&&ans!='Y')break;
   }
}

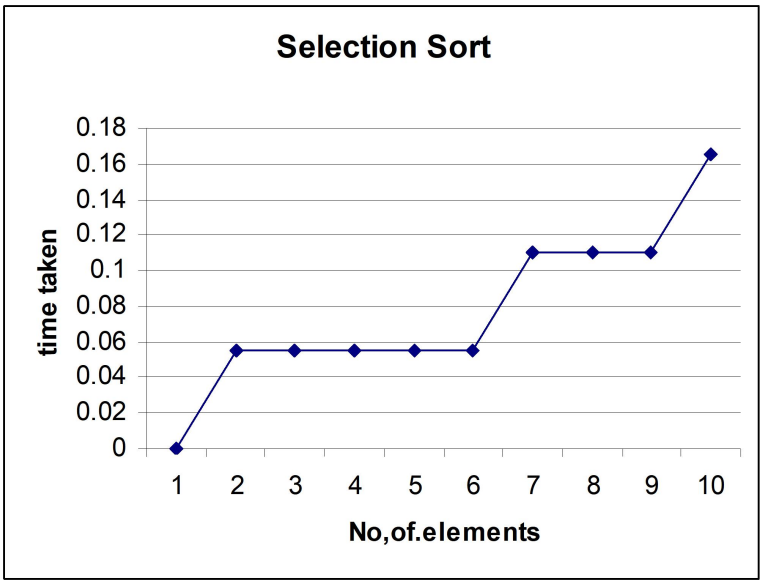
void selectionsort(int a[],int n)
{
   int i,j,min;
   for(i=0;i<n-1;i++)
   {
     min=i;
     for(j=i+1;j<n;j++)
          if(a[j]<a[min])
            min=j;
            swap(a,i,min);
    }
}

void swap(int a[],int i,int min)
{
   int temp;
   temp=a[i];
   a[i]=a[min];
   a[min]=temp;
}


/*Output
enter the no. of elements
5
the sorted elements are:
130     346     1090    10982   11656
the time taken for sorting is 0.000000 sec

like to have another solution(y/n)?:n */
```

GRAPH


Selection Sort
time taken
0.18
0.16
0.14
0.12
0.1
0.08
0.06
0.04
0.02
0
1
2
3
4
5
6
7
8
9
10
No,of.elements

Experiment No. 10 **Date: 28 /04/2026 __**

10. Design and implement C/C++ Program to sort a given set of n integer elements using Quick Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

Algorithm:

```
ALGORITHM Quick sort (A[l….r])
    // Sorts a sub array  by quick sort
    //Input : A sub array A[l..r] of A[0..n-1] ,defined by its left and right indices l
    //and r
    // Output : The sub array A[l..r] sorted in non decreasing  order
            if  l < r
               s = Partition (A[l..r])  //s is a split position
               Quick sort (A[l …s-1])
               Quick sort (A[s+1…r])

  ALGORITHM   Partition (A[l…r])
  //Partition a sub array  by using its first element as a pivot
  // Input : A sub array A[l…r] of  A[0…n-1]  defined  by its left and right indices l and // r  (l < r)
  // Output : A partition of  A[l…r], with the split position returned as this function's  value
  p=A[l]
  i=l;
  j=r+1;
  repeat
     delay(500);
     repeat  i= i+1 until A[i] >= p
     repeat  j=j-1  until  A[J] <= p
     Swap (A[i],A[j])
  until I >=j
  Swap (A[i],A[j])  // Undo last Swap when i>= j
  Swap (A[l],A[j])
  Return j
```

call clock( ) function to record the end time in terms of clock ticks after sorting

```
    end=clock();
```

Calculate the time in terms of seconds required to sort n elements using Quick sort i.e elapse time

```
   elapse_time = (end-start);
    elap se_time = (end-start)/CLK_TCK;
   Print  "elapse_time".
```

/*PROGRAM*/

```
#include<stdio.h>
#include<stdlib.h>
#include<conio.h>
#include<time.h>

void main()
{
   int i,n,a[25000];
   char ans=0;
   clock_t start,stop;
   void quicksort(int [],int,int);
   clrscr();
   for(;;)
   {
   printf("enter the no. of elements to be sorted\n");
   scanf("%d",&n);
//  printf("enter the elements to be sorted\n");
   for(i=0;i<n;i++)
// scanf("%d",&a[i]);
   a[i]=rand();
   start=clock();
   quicksort(a,0,n-1);
   stop=clock();
   printf("the sorted elements are:\n");
   for(i=0;i<n;i++)
   printf("%d\t",a[i]);
   printf("\nthe time taken for sorting is %f sec\n",(stop-start)/CLK_TCK);
   printf("\nlike to have another solution(y/n)?:");
   fflush(stdin);
   scanf("%c",&ans);
    if(ans!='y'&&ans!='Y')break;
   }
}
```

```
void quicksort(int a[],int j,int k)
{
  int m;
  int partition(int [],int,int);
  if(j<k)
  {
    m=partition(a,j,k);
    quicksort(a,j,m-1);
    quicksort(a,m+1,k);
  }
}
int partition(int a[],int p,int q)
{
 int i,m,temp;
 m=p++;
 while(1)
 {
   while(a[p]<=a[m])
   p++;

   while(a[q]>a[m])
   q--;

   if(p>=q) break;

   temp=a[p];
   a[p]=a[q];
   a[q]=temp;
 }
  temp=a[m];
  a[m]=a[q];
  a[q]=temp;
  return q;
}
```

Output

Sorting using Quick sort method

```
Enter the value of n:10
Enter the array elements:2 1 4 56 8 7 9 90 65 45
The sorted array is:1   2       4       7       8       9       45      56
65      90

Time=6.318681

Enter the value of n:5
Enter the array elements:12 11 78 98 56
The sorted array is:11  12      56      78      98

Time=2.362637

Enter the value of n:15
```

Enter the array elements:15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
The sorted array is:1 2 3 4 5 6 7 8
9 10 11 12 13 14 15

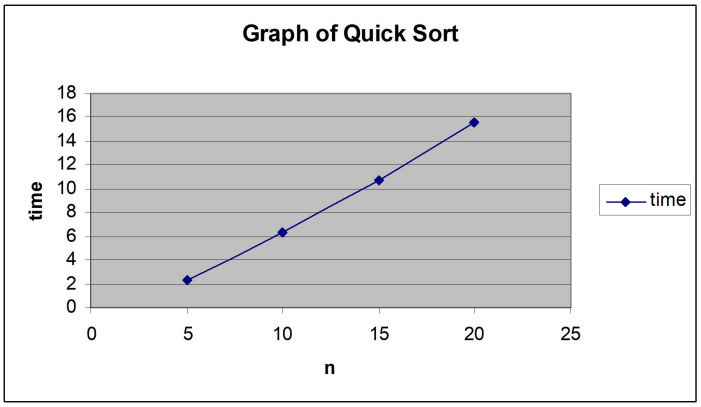
Time=10.714286

Enter the value of n:20
Enter the array elements:20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
The sorted array is:1 2 3 4 5 6 7 8
9 10 11 12 13 14 15 16 17 18
19 20

Time=15.604396

Graph

Quick Sort	
n	**time**
5	2.362
10	6.318
15	10.714
20	15.604



Experiment No. 11 **Date: 05_/05_/2026 _**

Design and implement C/C++ Program to sort a given set of n integer elements using Merge Sort method and compute its time complexity. Run the program for varied values of n> 5000, and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

ALGORITHM MERGESORT(A[0..N - 1])

```
ALGORITHM Mergesort(A[0..n - 1]
  //Sorts array A[0..n - 1] by recursive mergesort
  //Input: An array A[0..n - 1] of orderable elements
  //Output: Array A[0..n - 1] sorted in nondecreasing order
  if n > 1
        copy A[0..⌊n/2⌋ - 1] to B[0..⌊n/2⌋ - 1]
        copy A[⌊n/2⌋..n - 1] to C[0..⌈n/2⌉ - 1]
        Mergesort(B[0..⌊n/2⌋ - 1])
        Mergesort(C[0..⌈n/2⌉ - 1])
        Merge(B, C, A)
```

ALGORITHM Merge(B[0..p - 1], C[0..q - 1], A[0..p + q - 1])

```
ALGORITHM Merge(B[0..p - 1], C[0..q - 1], A[0..p + q - 1])
  //Merges two sorted arrays into one sorted array
  //Input: Array B[0..p - 1] and C[0..q - 1] both sorted
  //Output: Sorted array A[0..p + q - 1] of the elements of B and C
  I ← 0; j ← 0; k ← 0
  while I < p and j < q do
        if B[i] ≤ C[j]
            A[k] ← B[i]; i ← i + 1
        else A[k] ← C[j]; j ← j + 1
        k ← k + 1
  if i = p
     copy C[j..q - 1] to A[k..p + q - 1]
else  copy B[i..p - 1] to A[k..p + q - 1]
```

PROGRAM

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>

void mergesort(int [],int,int);

void main()
{
   int n,a[25000],key,pos,i;
   char ans=0;
   clock_t start,stop;
    for(;;)
   {
   printf("enter the no. of elements in the array\n");
   scanf("%d",&n);
//   printf("enter the elements in the array\n");
   for(i=0;i<n;i++)
//   scanf("%d",&a[i]);
   a[i]=rand();
   start=clock();
   mergesort(a,0,n-1);
   stop=clock();
   printf("the sorted elements are\n");
   for(i=0;i<n;i++)
   printf("%d\t",a[i]);
   printf("time taken is :%f sec\n",(stop-start)/CLK_TCK);

   printf("\nlike to have another solution(y/n)?:");
                fflush(stdin);
                scanf("%c",&ans);
   if(ans!='y'&&ans!='Y')break;
   }

}
```

```
void mergesort(int a[],int low,int high)
{
  int mid;
  void merge(int [],int,int,int);
  if(low<high)
  {
    mid=(low+high)/2;
    mergesort(a,low,mid);
    mergesort(a,mid+1,high);
    merge(a,low,mid,high);
  }
}

void merge(int a[],int low,int mid,int high)
{
  int b[25000],i1,i2,j,k;
  i1=low;
  i2=mid+1;
  j=low;
  while((i1<=mid)&&(i2<=high))
  {
    if(a[i1]<=a[i2])
    {
          b[j]=a[i1];
          i1++;

    }
    else
    {
          b[j]=a[i2];
          i2++;
    }
         j++;
  }
    if(i1>mid)
    for(k=i2;k<=high;j++,k++)
    b[j]=a[k];
    else
    for(k=i1;k<=mid;j++,k++)
    b[j]=a[k];
    for(k=low;k<=high;k++)
    a[k]=b[k];
}
```

Output

Sorting using Merge sort

Enter the size of first array:3

Enter the array element:3 4 1
The sorted array is:1 3 4
The time required for sorting is:1.98 secs

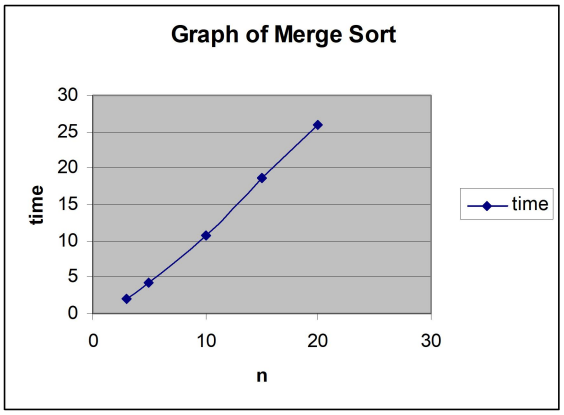
Enter the size of first array:5

Enter the array element:1 5 6 8 9
The sorted array is:1 5 6 8 9
The time required for sorting is:4.23 secs

Enter the size of first array:15

Enter the array element:2 4 3 1 6 5 8 7 9 10 13 12 11 14 15
The sorted array is:1 2 3 4 5 6 7 8
9 10 11 12 13 14 15
The time required for sorting is:18.74 secs

Merge Sort	
n	time
3	1.98
5	4.23
10	10.66
15	18.74
20	25.99



Experiment No. 12 **Date: 12_/05/2026 __**

.Design and implement C/C++ Program for N Queen's problem using Backtracking.

ALGORITHM N QUEENS

//Place n queens on an n*n board using back tracking method such that there are no attacks between any queens

//Input: No.of queens to be placed on an n*n board

//Output: Position of the queens on the chessboard

```
Board = N*N array
Top:
board(1, 1): = 1                                    // a queen is placed in top left
Call promising                                      //  see if any promising nodes
if promising = TRUE , board(x, y): = 1              // if promising place a queen
if promising  = TRUE x = x+1                        // goto next row
if n queens placed return true                      //  problem is solved
if row=1 and no queens placed, return false   // problem can't be solved
if no queens placed, begin:
            backtracking to previous row, erase queen
            have we tried all combo's?
            If  True, return false                  // problem can't be solved
column:=0
goto top                                            // continue loop until problem solved
                                                    //  or all combos tried
```

/***PROGRAM*/**

```
#include<stdio.h>
#include<math.h>
#define max 10
int x[max],count=0;
void writesolution(int n)
{
        int chessboard[max][max],i,j;
        count=count+1;
        for(i=1;i<=n;i++)
        for(j=1;j<=n;j++)
        chessboard[i][j]=0;
        for(i=1;i<=n;i++)
        chessboard[i][x[i]]=i;
        printf("the solution %d\n",count);
        for(i=1;i<=n;i++)
```

```
        {
                for(j=1;j<=n;j++)
                printf("%d\t",chessboard[i][j]);
                printf("\n");
        }
}
int place(int k, int i)
{
        int j;
        for(j=1;j<=k-1;j++)
        if((x[j]==i)||(abs(x[j]-i)==abs(j-k)))
        return 0;
        return 1;
}
void nqueens(int k,int n)
{
        int i,j;
        for(i=1;i<=n;i++)
        {
                if(place(k,i))
                {
                        x[k]=i;
                        if(k==n)
                        writesolution(n);
                        else
                        nqueens(k+1,n);
                }
        }
}



int main( )
{
        int n,i,j;
        printf("enter the number of queens\n");
        scanf("%d", &n);
        nqueens(1,n);
        if(count==0)
        printf("no solution for %d queens problem\n",n);
        return 0;
}
```

OUTPUT1

enter the number of queens
1
the solution 1
1

OUTPUT2

enter the number of queens
2
no solution for 2 queens problem

OUTPUT3

```
enter the number of queens
4
the solution 1
0       1       0       0
0       0       0       2
3       0       0       0
0       0       4       0
the solution 2
0       0       1       0
2       0       0       0
0       0       0       3
0       4       0       0
```

OUTPUT4

```
enter the number of queens to be placed in N*N square:4
[][*][][]
[][][][*]
[*][][][]
[][][*][]

like to have another solution(y/n)?:y
[][][*][]
[*][][][]
[][][][*]
[][*][][]

like to have another solution(y/n)?:y

 no more solution

 2 different solution are possible with 4 queen */
```

Sample Viva questions

ALGORITHMS LABORATORY
Viva Questions

1) What is the time complexity of linear search?
Θ(n)

2) What is the time complexity of binary search?
Θ($\log_2$n)

3) What is the major requirement for binary search?
The given list should be sorted.

4) What is binary search?

It is an efficient method of finding out a required item from a given list, provided the list is in order.

The process is:

1. First the middle item of the sorted list is found.
2. Compare the item with this element.
3. If they are equal search is complete.
4. If the middle element is greater than the item being searched, this process is repeated in the upper half of the list.
5. If the middle element is lesser than the item being searched, this process is repeated in the lower half of the list.

5) What is parental dominance?

The key at each node is greater than or equal to the keys at its children.

6) What is heap?

A **heap** can be defined as a binary tree with keys assigned to its nodes (one key per node) provided the following two conditions are met:
1 The tree's shape requirement – The binary tree is essentially complete (or simply complete), that is, all its levels are full except possibly the last level, where only some rightmost leaves may be missing.
2. The parental dominance requirement – The key at each node is greater than or equal to the keys at its children.

7) What is height of Binary tree?

It is the longest path from the root to any leaf.

8) What is the time complexity of heap sort?

Θ(n logn)

9) What is Merge Sort?

Merge sort is an O (n log n) comparison-based sorting algorithm.
Where the given array is divided into two equal parts. The left part of the array as well as the right part of the array is sorted recursively. Later, both the left part and right part are merged into

a single sorted array.

10) Who invented Merge Sort?

John Von Neumann in 1945.

11) On which paradigm is it based?

Merge-sort is based on the divide-and-conquer paradigm.

12) What is the time complexity of merge sort?

n log n.

13) What is the space requirement of merge sort?

Space requirement: Θ(n) (not in-place).

14) When do you say an algorithm is stable and in place?

Stable – if it retains the relative ordering.

In – place if it does not use extra memory.

15) Name the design strategy used in Selection Sort.

Brute force

16) **Define Brute force strategy.**

Brute force is a straight forward approach to solving a problem, usually directly based on the problem's statement and definitions of the concepts involved.

17) What is the time complexity of Selection Sort algorithm?

$\Theta(n^2)$

18) Is Selection Sort Inplace?

Yes.

19) Is Selection Sort stable?

Yes.

20) Why is the name Selection Sort?

Algorithm selects the smallest number in the array and places it in its final position the sorted array.

21) When is Selection Sort algorithm useful?

Selection Sort algorithm is useful for small inputs because it requires $\Theta(n^2)$ comparisons.

Selection Sort algorithm requires (n-1) swaps & hence Θ (n) memory writes. Thus it can be very useful if writes are the most expensive operation.

22) What are the drawbacks of Selection Sort?

Inefficient on large lists.

It requires same number of iterations no matter how well-organized the array is.

It uses internal sorting, means it would require the entire array to be in main memory.

23) When is a problem said to have overlapping sub problems?

A problem is said to have overlapping subproblems if it can be broken down into subproblems which are reused multiple times. This is closely related to recursion.

24) What is principle of optimality?

The optimal solution to any instance of an optimization problem is composed of optimal solutions to its subinstances.

25) Can knapsack problem be solved in any other technique?

It can be solved using many other techniques such as BruteForce, Greedy technique, Branch and bound etc..

26) Mention few applications of dynamic programming.

Can be applied to find factorial, fibonacci numbers in which the required value may depend on previously computed values.

To find the longest common substrings of the strings "ABAB", "BABA" and "ABBA" are the strings "AB" and "BA" of length 2. Other common substrings are "A", "B" and the empty string "". etc..

27) Compare Dynamic programming and Divide and conquer.

Dynamic programming differs from the "Divide and Conquer" (D-and-C) method in the fact that the problems solved by D-and-C can have many non-overlapping subproblems - i.e, new subproblems may be generated when the method is applied.

Both use recursion.

28) Who invented Dijkstra's Algorithm?

Edsger Dijkstra invented this algorithm.

29) What is the other name for Dijkstra's Algorithm?

Single Source Shortest Path Algorithm.

30) Which technique the Dijkstra's algorithm is based on?

Greedy Technique.

31) What is the purpose of Dijkstra's Algorithm?

To find the shortest path from source vertex to all other remaining vertices

32) Name the different algorithms based on Greedy technique.

Prim's Algorithm, Kruskal's algorithm

33) **What is the constraint on Dijkstra's Algorithm?**

This algorithm is applicable to graphs with non-negative weights only.

34) What is a Weighted Graph?

A weighted graph is a graph with numbers assigned to its edges. These numbers are called weights or costs.

35) What is a Connected Graph?

A graph is said to be connected if for every pair of its vertices u and v there is a path from u to v.

36) What is the time complexity of Dijkstra's algorithm?

For adjacency matrix, It is O(IVI*IVI)

For adjacency list with edges stored as min heap, it is O(IEIlogIVI)

37) Differentiate b/w Traveling Salesman Problem(TSP) and Dijkstra's Algorithm.

In TSP, given n cities with known distances b/w each pair , find the shortest tour that passes through all the cities exactly once before returning to the starting city.

In Dijkstra's Algorithm, find the shortest path from source vertex to all other remaining vertices

38) Differentiate b/w Prim's Algorithm and Dijkstra's Algorithm?

Prim's algorithm is to find minimum cost spanning tree.

Dijkstra's algorithm is to find the shortest path from source vertex to all other remaining vertices.

39) What are the applications of Dijkstra's Algorithm?

It finds its application even in our day to day life while travelling.

They are helpful in routing applications.

40) Who was the inventor of the Quicksort?

C.A.R.HOARE, a British Scientist.

41) Which design strategy does Quicksort uses?

Divide and Conquer

42) What is Divide and Conquer Technique?

(I) Divide the instance of a problem into two or more smaller instances
(II) Solve the smaller instances recursively
(III) Obtain solution to original instances by combining these solutions

43) What is the another name for Quicksort?

Partition Exchange Sort

44) What are the advantages and disadvantages?

Advantage: Fastest among all sorting algorithms Suitable when the input size is very large
Disadvantage:
Heavily based on recursive sorting technique
It takes O (n* log n) time and O (log n) additional space due to Recursion

45)Is Quicksort Stable as well as In place?

Not Stable but In place.

46) When do you say that a Quick sort having best case complexity?

When the pivot exactly divides the array into equal half

47) When do you say that Quick sort having worst case complexity?

When any one of the subarray is empty

48) How many more comparisons are needed for average case compared to best case?

38% more

49) when do you say that the pivot element is in its final position?

When all the elements towards the left of pivot are <= pivot and all the elements towards the right of pivot are >= pivot.

50) What is CLK_TCK?

macro defines the number of clock ticks per second which is available in the TIME.H header file.

51) What technique does kruskal's algorithm follow.

greedy technique

52) What is the time complexity.

With an efficient sorting algorithm, the time efficiency of an kruskal's algorithm will be in O(|E|log|E|).

53) Who is the inventor of kruskal's algorithm.

Joseph Kruskal.

54) Which technique is used to solve BFS & DFS problems?

Decrease and Conquer

55) What is BFS traversal?

It proceeds in a concentric manner by visiting first all the vertices that are adjacent to a starting vertex, then all unvisited vertices two edges apart from it and so on , until all the vertices are in the same connected component as the starting vertex are visited.

If there still remains unvisited vertices, the algorithm to be restarted at an arbitrary vertex of another connected component of a graph.

56) What is DFS traversal?

Consider an arbitrary vertex v and mark it as visited on each iteration, proceed to an unvisited vertex w adjacent to v. we can explore this new vertex depending upon its adjacent information. This process continues until dead end is encountered.(no adjacent unvisited vertex is available). At the dead end the algorithm backs up by one edge and continues the process of visiting unvisited vertices. This process continues until we reach the starting vertex.

57) What are Tree edge, Back edge & Cross edge ?

Tree edge: whenever a new unvisited vertex is reached for the first time, it is attached as a child

to the vertex from which it is being visited. Such an edge is called a tree edge.

Back edge: An edge leading to a previously visited vertex other than its immediate predecessor. Such an edge is called a back edge

Cross edge If an edge leading to a previously visited vertex other than its immediate predecessor (i.e. its parent in the tree). Is encountered, the edge is noted as a cross edge.

58) What are the various applications of BFS & DFS tree traversal technique?

Application of BFS

Checking connectivity and checking acyclicity of a graph.

Check whether there is only one root in the BFS forest or not.

If there is only one root in the BFS forest, then it is connected graph otherwise disconnected graph.

For checking cycle presence, we can take advantage of the graph's representation in the form of a BFS forest, if the latter vertex does not have cross edge, then the graph is acyclic.

Application of DFS

To check whether given graph is connected or not.

To check whether the graph is cyclic or not.

To find the spanning tree.

Topological sorting.

59) Which data structures are used in BFS & DFS tree traversal technique?

We use Stack data structure to traverse the graph in DFS traversal.

We use queue data structure to traverse the graph in DFS traversal.

60) What are the efficiencies of BFS & DFS algorithms for Adjacency matrix and adjacency list representation?

Adjacency matrices: $\Theta(V^2)$

Adjacency linked lists: $\Theta(V+E)$

61) Define set?

Set is unordered collection of distinct items.

62) Define power set?

The set of all subsets of a set is a power set.

63) Define squashed order?

Any subsets involving aj can be listed only after all the subsets involving a1,…,aj-1(j=1,2,…,n-1)

64) What are the ways of generating subsets?

Decrease by constant

Bit string based algorithm

Minimal change algorithm

65) Explain time and space trade off strategy.

It is a strategy in which time efficiency can be achieved at the cost of extra memory usage.

66) Which technique does the Horspool algorithm uses?

Input enhancement technique

67) Explain Input enhancement technique.

It is a technique in which the problem's input is preprocessed in whole or in part and the additional information stored is obtained to solve the problem.

68) What is a shift table?

It is a table that stores the shift information i.e it stores the distance of each rightmost character in the first "m-1" characters of the pattern from the last character of the pattern and stores the length of the pattern as the shift size if the character is not present in the pattern.

69) Which is better Brute Force or Horspool algorithm for string matching?

Horspool is better as it reduces the number of comparisons.

70) Differentiate between Brute-Force technique and Horspool algorithm for string

matching.

a) In brute-force, comparison starts from left where as in Horspool comparison starts from right.

b) In brute-force technique, if there is a character mismatch then we shift the pattern to the right by 1 position.

In Horspool, if there is a mismatch then the shift size is obtained from the shift table and the pattern is shifted accordingly.

71) Differentiate between Horspool and Boyer -Moore algorithms.

Horspool algorithm uses only Shift table to obtain the shift information.

Boyer - Moore algorithm uses 2 tables.

-Bad symbol shift table (same as the shift table used by Horspool).The shift information of this table is known as d1

-Good suffix shift table

The shift information from this table is known as d2.

Shifting is done by taking max(d1,d2).

72) What is Dynamic Programming?

It is a technique for solving problems with overlapping sub problems.

73) What does Dynamic Programming have in common with Divide-and-Conquer?

Both solve the problems by dividing the problem into sub problems. Using the solutions of sub problems, the solutions for larger instance of the problem can be obtained.

74) What is the Space & Time complexity of DP algo for Binomial Coefficient C(n, k)?

Θ(nk)

75) What is a principle difference between the two techniques?

Only one instance of the sub problem is computed & stored. If the same instance of sub problem is encountered, the solution is retrieved from the table and never recomputed.

Very precisely re - computations are not preformed.

76) What is a spanning tree ?

A spanning tree of a weighted connected graph is its connected acyclic(no cycles) subgraph (i.e. a tree)that contains all the vertices of a graph.

77) What is a minimum spanning tree ?

Minimum spanning tree of a connected graph is its spanning tree of smallest weight.

78) Prim's algorithm is based on which technique.

Greedy technique.

79) State greedy technique for solving problem.

A greedy approach constructs a problem through a sequence of steps each expanding a partially constructed solution obtained so far, untill the complete solution is reached.

On each step the choice must be made as

Feasible: i.e. it has to satisfy the problem's constraints.

Locally optimal: i.e. it has to be the best local choice among all feasible choices available on that step.

Irrevocable: i.e. once made, it cannot be changed on subsequent steps of the algorithm.

What are fringes and unseen vertices?

Fringes: vertices which are connected are called fringes.

Unseen vertices : vertices which are not adjacent are called unseen.

80) Name some of the application greedy technique

They are helpful in routing applications: In the case of network where large number of computers are connected, to pass the data from one node to another node we can find the shortest distance easily by this algorithm.

Communication Networks: Suppose there are 5 different cities and we want to establish computer connectivity among them, then we take into the account of cost factor for communication links.

Building a road network that joins n cities with minimum cost.

81) Does Prim's Algorithm always yield a minimum spanning tree ?

Yes

82) What is the purose of Floyd's algoreithm?

- To find the all pair shortest path.

83) What is the time efficiency of floyd's algorithm?

- It is same as warshall's algorithm that is $\theta(n^3)$.

84) What is distance matrix?

- In which the element d_{ij} indicates the length of the path from i^{th} vertex to the jth vertex.

85) What is shortest path?

- It is the path which is having shortest length among all possible paths.

86) warshall's algorithm is optimization problem or non-optimization problem ?

Non-optimization problem

87) For what purpose we use Warshall's algorithm?

Warshall's algorithm for computing the transitive closure of a directed graph

88) Warshall's algorithm depends on which property?

Transitive property

89) what is the time complexity of Warshall's algorithm?

Time complexity of warshall's algorithm is $\Theta(n^3)$

90) what is hard problem?

Optimization problems which are having complexity of exponential or factorial.

91) hard problems can be solved by which method?

Back tracking and branch and bound techniques used for large instance of combinatorial problems.

92) what is state space tree?

Constructing a tree of choices being made and processing is known as state-space-tree. Root represents an initial state before the search for solution begins.

93) what are promising and non-promising state-space-tree?

Node which may leads to solution is called promising.

Node which doesn't leads to solution is called non-promising.

94) Differentiate between backtracking and Branch and Bound.

Branch and Bound uses Best first search technique and is applicable to optimization problems.

Backtracking is applicable to non-optimization problems most of the times and sometimes optimization problems. It uses DFS search technique.

95) In Backtracking what does the leaf node indicates?

It indicates either a non promising dead ends or solutions to the problem.